Seminarium dyplomowe, 3

Układ aktywnego tłumienia hałasu pod nadzorem jądra systemu Linux z rozszerzeniami real-time

Daniel Kopiecki

Politechnika Śląska

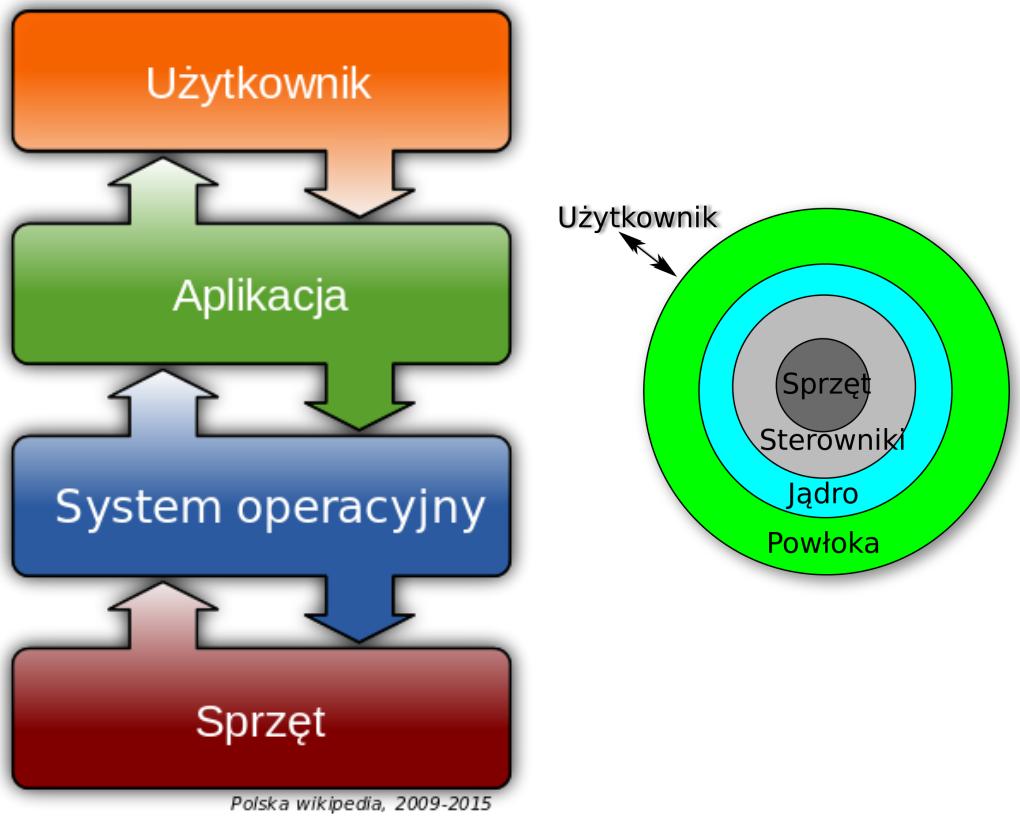
29 maja 2023

Słowem wstępu - system operacyjny

- System operacyjny w najprostszej definicji to oprogramowanie zarządzające systemem komputerowym.
- Z punktu widzenia użytkownika, stanowi środowisko, pozwalające na interakcję z komputerem bez zrozumienia i znajomości działania urządzenia na poziomie sprzętowym.
- W uogólnieniu system operacyjny zajmuje się:
  1. Planowanie i przydziałem czasu procesora na wykonanie poszczególnych zadań (programów).
  2. Kontrolą i przydziałem pamięci na zadania (programy).
  3. Synchronizacją zadań (programów).
  4. Obsługą zasobów sprzętowych, zapewnieniem jednolitego interfejsu kontroli nad nim.

Słowem wstępu - system operacyjny

- Przyjęto, że system operacyjny dzieli się na trzy elementy:
  - **Jądro** - kontrola zadań i zasobów (sprzęt, pamięć, czas procesora).
  - **Powłoka** - interakcja użytkownika z jądrem.
  - **System plików** - sposób strukturyzacji danych na nośniku.

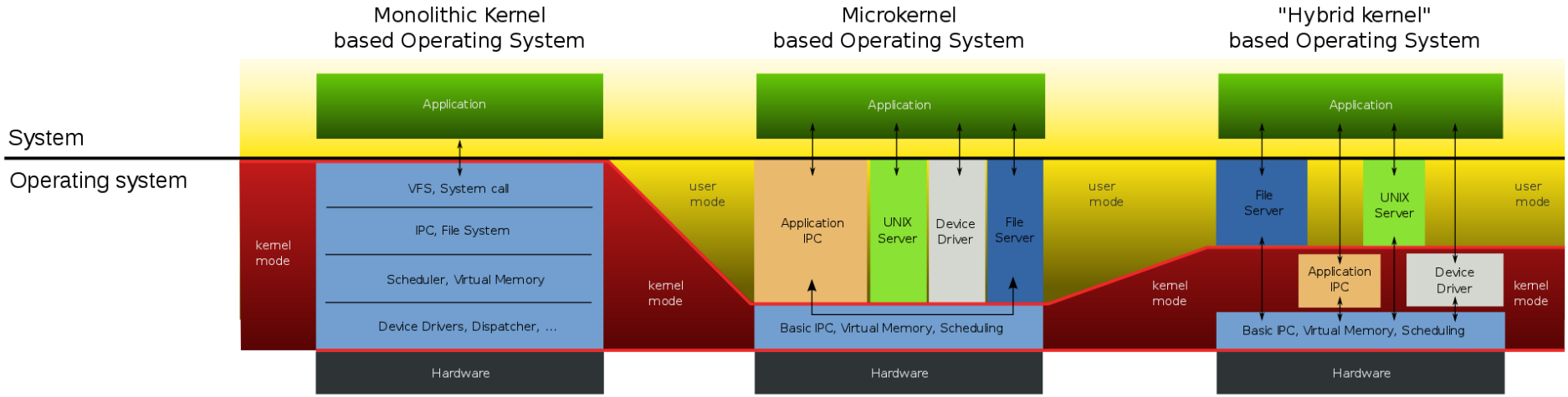


Rysunek: System operacyjny

Jądro systemu operacyjnego

- Jądro systemu operacyjnego to program, który posiada pełną kontrolę nad wszystkim co dzieje się w systemie komputerowym.
- Kod tego programu jest umiejscowiony w odpowiednim, chronionym przed zapisem obszarze pamięci, a ładowany jest przy uruchomieniu systemu z użyciem **bootloader'a**.
- Istnieją różne architektury jądra, z których wyróżnić można:
  - **Monolityczne** - jądro stanowi jeden, duży program (systemy uniksopodobne, lecz nie tylko) pracujący w **trybie jądra**.
  - Mikrojądro - monolit zostaje podzielony na mniejsze moduły, z których część pracuje w **trybie użytkownika** komunikujac się z serwerami (pośrednikami).
  - Nanojądro - jeszcze większe podzielenie na moduły.
  - Hybrydowe - kompromis pomiędzy monolitem, a modularyzacją.

Jądro systemu operacyjnego

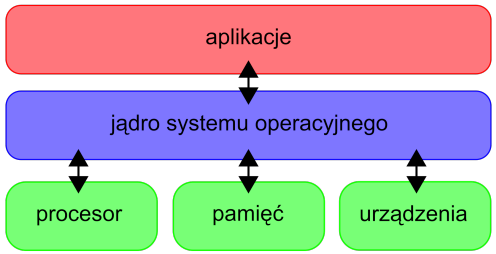


Rysunek: Schematy interakcji

- Monolit - jądro UNIX (np. BSD), **Linux**, MS-DOS
- Mikro/nano - QNX, Mach
- Hybryda - Windows NT/2000/XP/Vista/7/8/10/11

Jądro monolityczne

- Cechuje się wysoką wydajnością, ponieważ jedynym pośrednikiem między zasobami, a aplikacjami, jest właśnie jądro.
- Jądro zapewnia względnie wysokopoziomowe API do interakcji z zasobami (właściwie samoistnie je definiuje).
- Interakcje z jądrem odbywają się poprzez wywoływania systemowe (ang. *system call*).



Rysunek: Schemat interakcji - jądro monolityczne

- Dodatkową możliwością w tego typu architekturze jest dodawanie sterowników jądra jako ładowalnych modułów (ang. *loadable kernel modules*).

Łamanie trybu użytkownika (*breaking user's space*)

- Rozwiązaniem krytykowanym przez dużą rzeszę programistów jest oddzielanie kwestii systemowych do pracy w trybie użytkownika (jądra hybrydowe).
- Tego typu rozwiązanie nie dość, że powoduje przeogromne problemy z zachowaniem kompatybilności wstecznej, to jeszcze całkowicie uzależnia programistów pracujących pod danym systemem operacyjnym od jego wersji.
- W słowach Linusa Torvaldsa (LKML)
  Breaking user programs simply isn't acceptable. (. . .) We know that people use old binaries for years and years, and that making a new release doesn't mean that you can just throw that out. You can trust us.
- Przykładem może być stara gra komputerowa (zazwyczaj zaprojektowana do działania pod pewnym systemem operacyjnym), której uruchomienie dzisiaj jest wyjątkowo trudne z powodu braku kompatybilności.

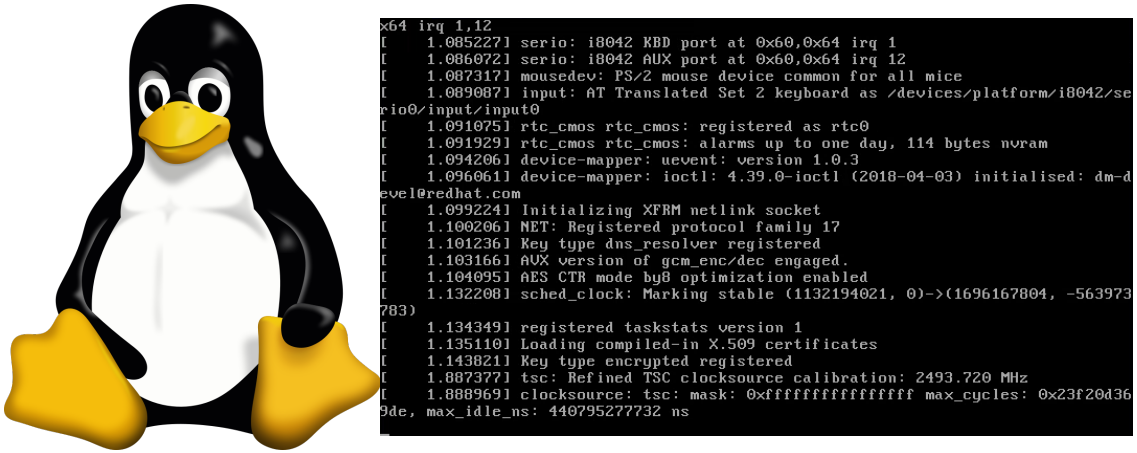
POSIX

- POSIX (ang. *Portable Operating System Interface for UNIX*, IEEE 1003) to standard oprogramowania systemu operacyjnego, w który skład wchodzi:
  - Interfejs programistyczny (<*unistd.h*>, read, write, fork, pipe...).
  - Interfejs użytkownika, przykładowo polecenia powłoki *echo, cat, grep*.
  - Właściwości powłoki.
- Pojęcie powstało na wskutek prób standaryzacji różnych odmian systemu operacyjnego UNIX.



Jądro Linux

- Linux to jądro systemu operacyjnego stworzone początkowo przez Linusa Torvaldsa w 1991 roku.
- Jest oparte na licencji GNU General Public License v2 i początkowo stanowiło wolny odpowiednik systemu UNIX.
- Jest jądrem monolitycznym i uniksopodobnym (jest w większości zgodne ze standardem POSIX).

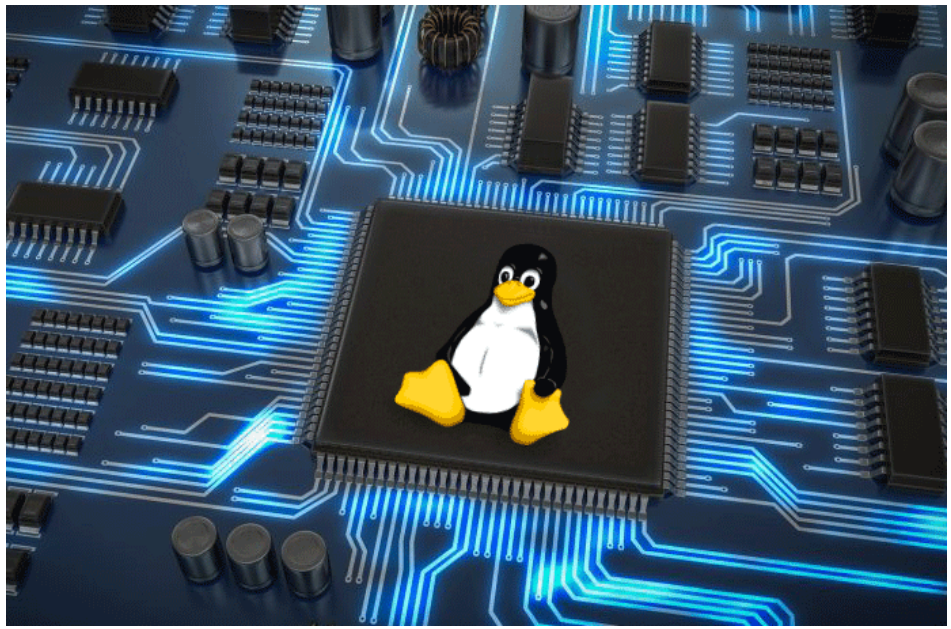


Rysunek: Jądro systemu Linux

- W połączeniu z narzędziami i bibliotekami systemowymi z projektu GNU, otrzymuje się kompletny system operacyjny **GNU/Linux**.

Wbudowany GNU/Linux

- Z powodu wszechstronności i otwartych kodów źródłowych, system operacyjny GNU/Linux jest szeroko wykorzystywany w systemach wbudowanych.
- Kod źródłowy może być portowany na różne architektury.
- Z racji wszechostronności systemu system można modyfikować usuwając i dodawając moduły.
- Pozwala to na tworzenie dedykowanych systemów operacyjnych stworzonych na potrzeby tworzonych urządzeń.



Wbudowany GNU/Linux - ograniczenia

- System GNU/Linux nie jest systemem czasu rzeczywistego.
- Jest to znaczące ograniczenie, przeszkadzające w wykorzystaniu systemu do urządzeń realizujących zadania krytyczne, gdzie wymagany jest w pełni deterministyczny czas realizacji.
- Powstało jednak sporo projektów skupiających się na przybliżeniu/dostosowaniu systemu GNU/Linux do systemu czasu rzeczywistego.
- Jednym z najczęściej wykorzystywanych rozwiązań jest projekt **Real-Time Linux**, znany ściślej jako zestaw łatek (ang. *patch*) jądra o nazwie **PREEMPT_RT**.



System czasu rzeczywistego, RTC (ang. *Real-time computing*)

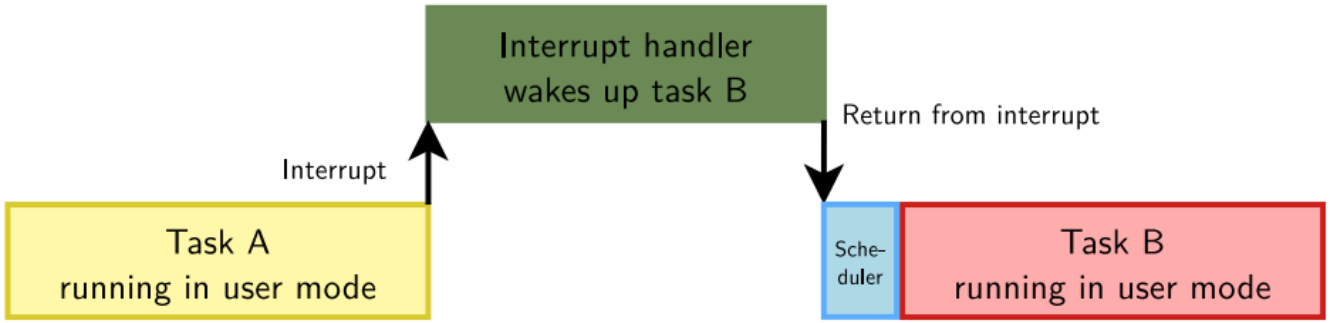
- Aby można stwierdzić było, że system jest systemem czasu rzeczywistego, poza logiczną prawidłowością wykonanych akcji, musi on również gwarantować określony czas ich realizacji.
- Zważając na określony czas realzacji, systemy czasu rzeczywistego przyjęło się dzielić na:
  - Hard real-time - przekroczenie limitu czasowego traktowane jest jak awaria systemu.
  - Firm real-time - przekroczenie limitu czasowego skutkuje dyskredytacją wykonywanego zadania.
  - Soft real-time - przekroczenie limitu czasowego skutkuje obniżeniem priorytetu wykonywanego zadania (ang. *usefulness of a result degrades*).
- W przypadku systemów jednozadaniowych sprawa jest trywialna, jednak komplikuje się, kiedy w grę wchodzi wielozadaniowość.
- Systemy operacyjne są wielozadaniowe.

Planowanie i wywłaszczenie

- Jądro systemu Linux implementuje **planistę** (ang. *scheduler*), którego zadaniem jest szeregowanie zadań w systemie (procesy, wątki).
- Tryb pracy planisty w jądrze Linux może być wybrany przez użytkownika. Domyślnym jest CFS (ang. *Completely Fair Scheduler*) i jego wybranie wynika z deklaracji zadania jako SCHED_NORMAL.
- Zadania są uruchamiane na podstawie ich priorytetu. Planista iteruje po obsługiwanych zadaniach i w pierwszej kolejności uruchamia te z największym priorytetem.
- Jądro systemu Linux zapewnia **wywłaszczenie zadania** z poziomu przestrzeni użytkownika.

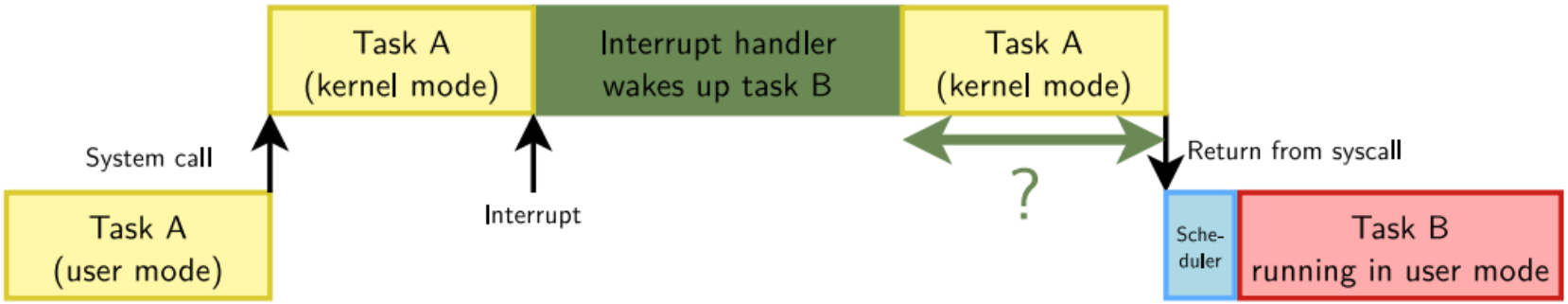
Wywłaszczenie

- **Wywłaszczenie** (ang. *preemption*) to akcja tymczasowego zatrzymania aktualnie wykonywanego zadania, mająca na celu wykonanie innego, zazwyczaj o większym priorytecie.
- Po wykonaniu zadania przerywającego, następuje powrót do wczesniej wykonanywanego. Akcję zatrzymania nazywa się **przerwaniem**.



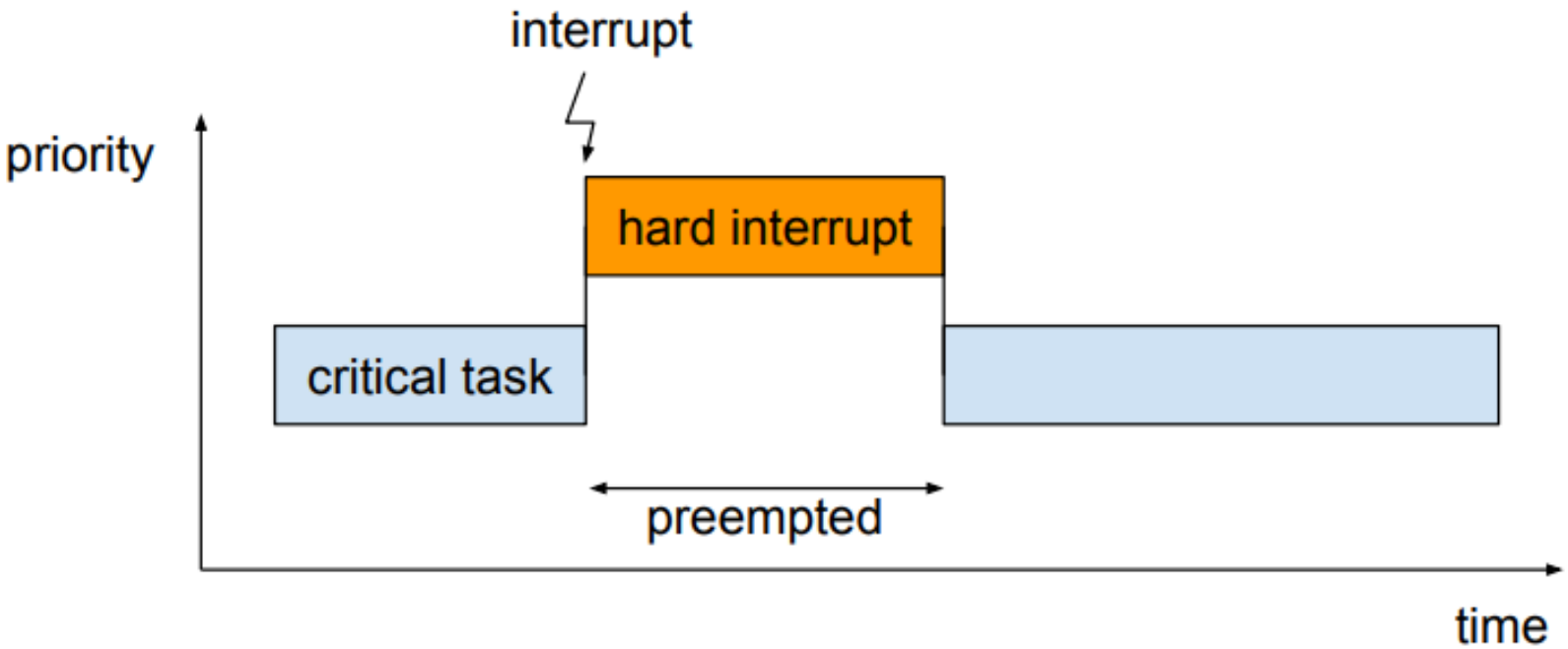
Wywłaszczenie

- W jądrze systemu Linux domyślnie wywłaszczone mogą zostać jedynie zadania pochodzące z przestrzeni użytkownika.
- Oznacza to, że jeżeli przerwanie nadejdzie w trakcie wykonywania przez zadanie wywołania systemowego, to takie przerwanie nie zostanie obsłużone do momentu zakończenia zawołania systemowego.



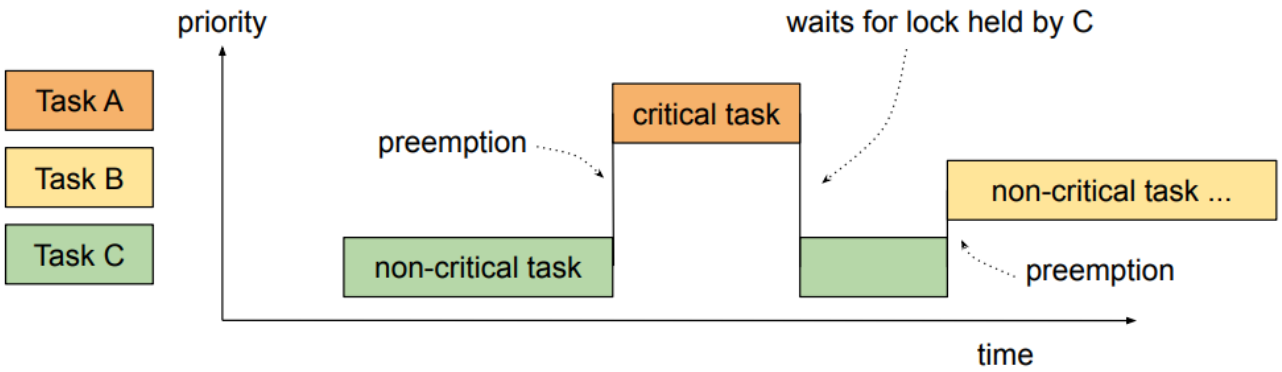
Wywłaszczenie

- Dodatkowo, każde zadanie z przestrzeni użytkownika może zostać wywłaszczone przez przerwanie sprzętowe. W trakcie obsługi takiego przerwania zablokowana jest obsługa innych.
- Oznacza to, że mało ważne przerwania sprzętowe mogą wywłaszczyć ważne, krytyczne czasowo zadania czasu rzeczywistego.



Wywłaszczenie

- Jakby tego było mało, istnieje jeszcze szereg problemów z synchronicznym dostępem do zasobów.
- Może zaistnieć sytuacja, kiedy niekrytyczne zadanie blokuje (ang. *mutex*) dostęp do zasobów zadaniu krytycznemu.



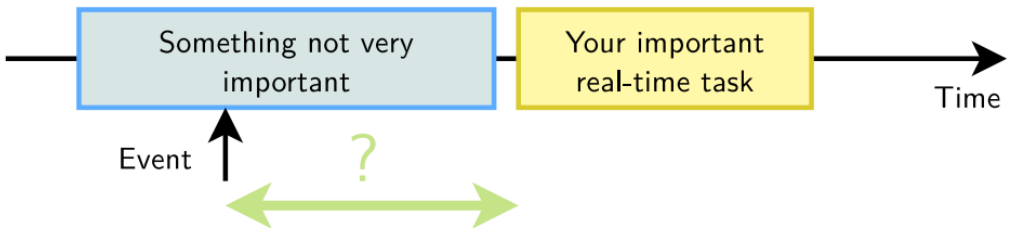
- Krytyczne zadanie A musi uzyskać dostęp do zasobów blokowanych przez zadanie C (niski priorytet).
- Planista więc uruchamia zadanie C, aby odblokować dostęp.
- W tym czasie zadanie C zostaje wywłaszczone przez zadanie B o wyższym priorytecie, ale niekrytyczne.
- Taki problem nosi nazwę **inwersji priorytetów** (ang. *priority inversion*).

Projekt PREEMPT_RT

- Zestaw łatek PREEMPT_RT celuje w możliwości wywłaszczenia wszystkich zadań realizowanych w systemie, nawet tych wykonywanych z przestrzeni jądra.
- Jako, że jądro Linux umożliwia korzystania z innych planistów niż domyślne, istnieje możliwość zastąpienia domyślnych wytycznych planisty na wspieranie rozszerzeń czasu rzeczywistego.
- Jako, że projekt PREEMPT_RT oferuje jedynie zestaw łatek, interfejs programistyczny POSIX/Linux pozostaje niezmienny.
- Umożliwa to korzystanie z systemu w tradycyjny sposób jak i wykonywanie krytycznych zadań równolegle w opraciu o planistę z rozszerzeniami czasu rzeczywistego.

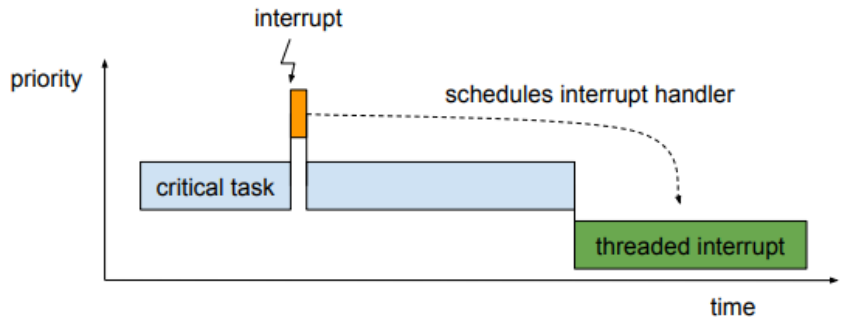
Projekt PREEMPT_RT

- Kilka funkcji łatkek PREEMPT_RT:
  - Wywłaszczenie zadań z przestrzeni jądra.
  - Liczniki wysokiej rozdzielczości (ang. *High resolution timers*).
  - Wątkową realizację przerwań sprzętowych.
  - Implementację blokad w trybie czasu rzeczywistego (do synchronizacji zadań).
  - Planistę opartego o algorytm Earliest-Deadline First.
  - Rozwiązanie problemu inwersji priorytetów poprzez zastosowanie **dziedziczenia priorytetu**.
- Jako, że czas wykonania zadania jest bardzo trudny do predykcji, celem jest minimalizacja latencji pomiędzy reakcją na wykonanie zadania.

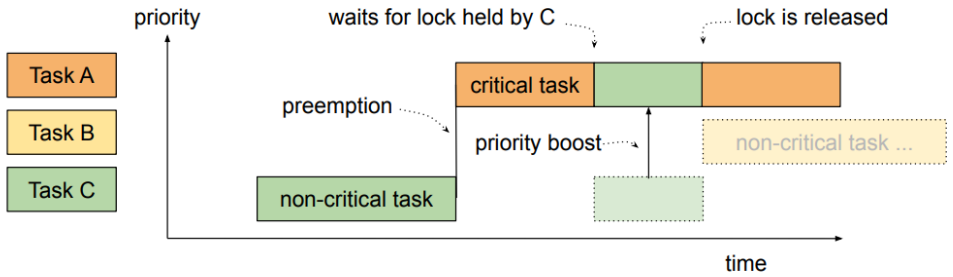


Projekt PREEMPT_RT

- Wątkowa realizacja przerwań sprzętowych.

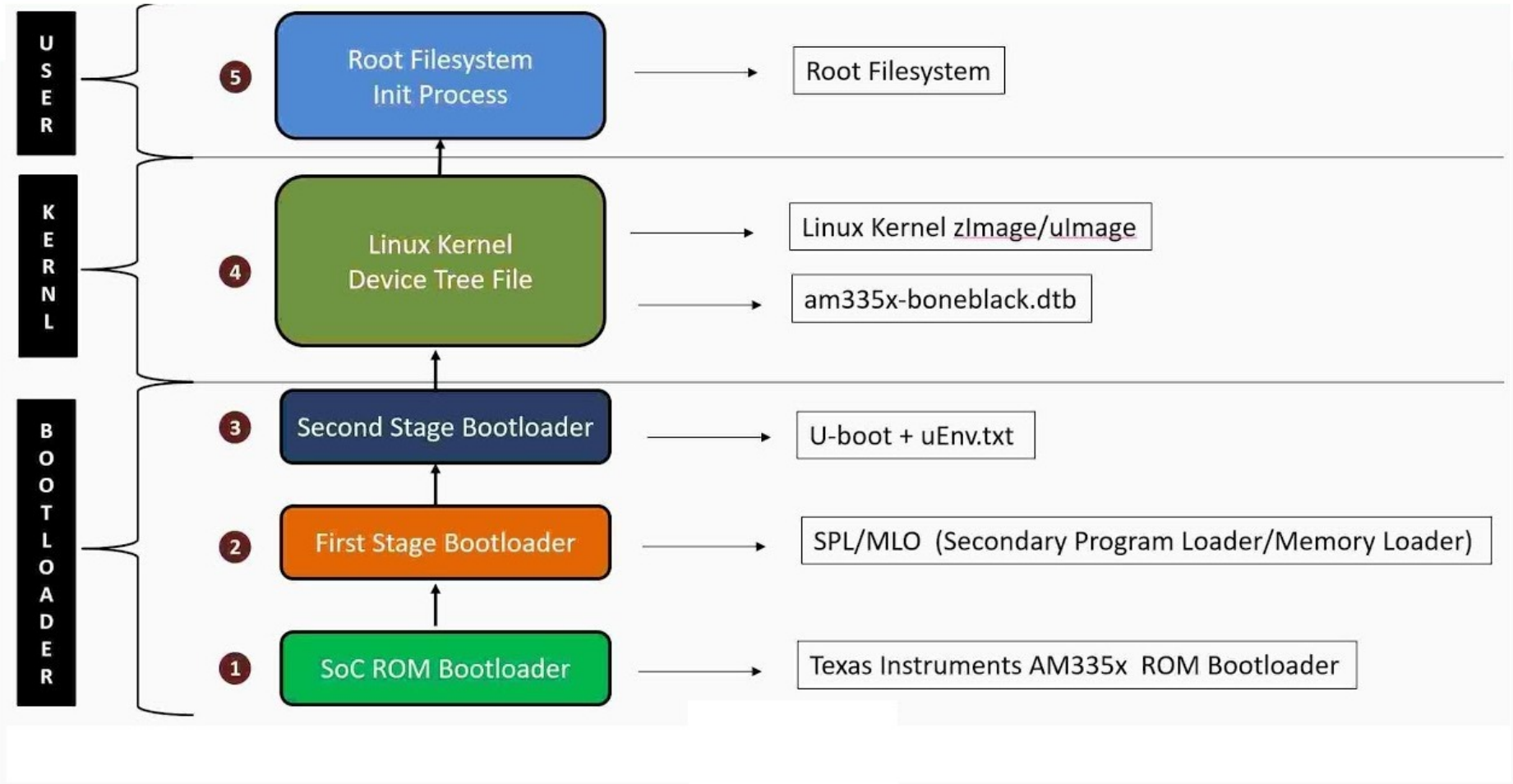


- Dziedziczenie priorytetu.



Krytyczne zadanie A musi uzyskać dostęp do zasobów blokowanych przez zadanie C. W związku z tym, priorytet zadania C jest zwiększany do priorytetu zadania A.

Embedded GNU/Linux, PREEMPT_RT - Proces uruchamiania, kolejność



Embedded GNU/Linux - przygotowywanie nośnika

```
sudo parted --script /dev/kartaSD mklabel msdos \
    mkpart primary fat32 0% 100MiB \
    mkpart primary ext4 100MiB 1100MiB \
    mkpart primary ext4 1100MiB 100% \
    set 1 boot on

sudo mkfs.vfat /dev/kartaSD1 -n uboot
sudo mkfs.ext4 /dev/kartaSD2 -L kernel
sudo mkfs.ext4 /dev/kartaSD3 -L rootfs
```

Embedded GNU/Linux, PREEMPT_RT - bootloader

- Das U-Boot (ang. *the Universal Boot Loader*) - główny, otwartoźródłowy bootloader używany w zastosowaniach embedded GNU/Linux.




```
U-Boot 2010.12-xes_r3 (Aug 25 2011 - 11:04:04)

CPU0:  P2020E, Version: 1.0, (0x80ea0010)
Core:  E500, Version: 4.0, (0x80211040)
Clock Configuration:
       CPU0:1066.667 MHz, CPU1:1066.667 MHz,
       CCB:533.333 MHz,
       DDR:400  MHz (800 MT/s data rate) (Asynchronous), LBC:133.333 MHz
L1:    D-cache 32 kB enabled
       I-cache 32 kB enabled
Board: X-ES XPedite5501 PMC/XMC SBC
       Rev SA, Serial# 36093001, Cfg 90015130-1
I2C:   ready
DRAM:  2 GiB (DDR3, 64-bit, CL=6, ECC on)
FLASH: Executed from FLASH1
POST memory PASSED
FLASH: 256 MiB
L2:    512 KB enabled
NAND:  4096 MiB
PCIE1: connected as Root Complex (no link)
PCIE1: Bus 00 - 00
PCIE2: disabled
PCIE3: disabled
In:    serial
Out:   serial
Err:   serial
DTT:   37 C local / 59 C remote (adt7461@4c)
DTT:   37 C local (lm75@48)
Net:   eTSEC1 connected to Broadcom BCM5482S
eTSEC2 connected to Broadcom BCM5482S
eTSEC1, eTSEC2
POST i2c PASSED
Hit any key to stop autoboot:  0
=>
```

- Kompilacja i kofiguracja bootloader'a:

```
$ git clone --depth 1 https://github.com/u-boot/u-boot.git
$ cd u-boot
$ make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- am335x-evm-defconfig -j5
$ make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- -j5
$ cp MLO /media/user/uboot/
$ cp u-boot.img /media/user/uboot/
```

Embedded GNU/Linux, PREEMPT_RT - jądro i system plików

- Kompilacja i konfiguracja jądra:

```
$ git clone --depth 1 https://github.com/beagleboard/linux.git 5.10.162-ti-rt-r59
$ cd linux
$ make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- bb.org_defconfig -j5
$ make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- -j5
$ cp arch/arm/boot/zImage /media/user/kernel
$ cp arch/arm/boot/dts/ am335x-boneblack.dtb /media/user/kernel
```

- Rozpakowanie gotowego systemu plików:

```
$ mkdir -p rootfs
$ cd rootfs
$ wget https://www.rcn-ee.com/rootfs/eewiki/minfs/ubuntu-20.04.5.tar.xz
$ tar -xf ubuntu-20.04.5-minimal-armhf-2022-12-20.tar.xz
$ rm ubuntu-20.04.5-minimal-armhf-2022-12-20.tar.xz
$ cp -r * /media/user/rootfs
```

Materiały źródłowe

- https://www.linux.org/
- https://bootlin.com/
- https://archive.kernel.org/oldwiki/rt.wiki.kernel.org/
- https://wiki.linuxfoundation.org/realtime/start

Dziękuję za uwagę